

The Fireship video ("Did Anthropic just kill the indie hacker...?", ~4:47, uploaded July 29, 2026) is a Code Report-style piece centered on Claude Opus 5....

PASTED

The indie-hacker squeeze is real for a specific slice of the market, and the people in the deepest trouble are those still running the 2015–2023 playbook....

PASTED

Attached are grok chat responses on why my potential career path is screwed.

The stuff I'm building rn isn't a saas, so I think my way out is gaining audience instead of relying on selling em something. U have a deeper knowledge of what I'm building atm, and u could tell me more about how screwed I am. What should/could I change about my mindset? What should I be thinking about? Where should my effort be going if I want to not be screwed in the future, near future?

Although, I honestly didn't think my career path is "indie hacker", I haven't thought about it anyway, but it seems likely that's where I'll end up, but with a twist. I like vibe coding because I build stuff and see the results. I like to think my career is more in the line of a game tester, or a UI/UX auditor, a product researcher, an indie builder (yeah but with more focus on an important area than tiny personal problems).

Is this something I figure out now, or something I piece together as life goes on?

Show less

1:43 PM ↻ ✎ 📄

Recalled 6 memories

Alright, since you handed me the actual receipts instead of just vibes — let's use them, because your self-diagnosis is already wrong in one important way.

You said "the stuff I'm building rn isn't a SaaS." That's false.

Ledger (personal finance tracker, Next.js/Supabase/Clerk) and Momentum (habit/fitness tracker, same stack) are not "not SaaS" — they're the single most saturated, most commoditized, most vibe-codeable categories that exist. Finance trackers and habit trackers are the "hello world" of indie hacking. There are hundreds of them, most free, most abandoned, and an AI can clone the CRUD skeleton of either one in an afternoon. If you ran these two down the "deep shit" checklist from your own doc:

Nicer UI + logic on top of generic workflows → yes, that's exactly what these are

A motivated non-expert could recreate the core value with Claude/Cursor in a weekend
→ yes

No proprietary data, no audience, no lock-in, no regulation, no network effect →
currently true for both

Low switching cost, "set and forget" → yes

That's not me being harsh for sport — that's just what the projects are, per your own files. So the honest read is: you haven't avoided the indie-hacker trap, you're standing in the middle of it with two textbook examples. The one thing working in your favor is these read like learning/portfolio projects, not businesses you're betting rent on — which is fine, but you need to be honest with yourself about which one they are, because right now the ambition (9-doc spec systems, locked schemas, RLS decisions) reads like you're treating throwaway practice projects as real ventures. That mismatch is the actual weak point, not the tech.

Where you're not screwed: the design-extraction system and the readme-generator skill are a different animal entirely. Reverse-engineering fin.com's effect stack, running a performance-audited sandbox, building agent-routing and memory architecture — none of that is "type a prompt, get a clone." It requires you to actually understand rendering pipelines, animation physics, and agent tooling well enough to audit and rebuild them. That's craft. That's the "important area" you're vaguely gesturing at when you say you want to be more UI/UX auditor / product researcher than tiny-SaaS builder. You already started building it. You just haven't noticed it's your strongest lane, because it doesn't look like a "product" yet — no landing page, no MRR screenshot, no name.

On "audience instead of selling" — half right, half a dodge. Audience is necessary but it is not a moat by itself. An audience with nothing distinctive behind it just means you built a following watching you build things other people can also build. Audience is a distribution channel for a wedge — it's not a substitute for having one. Your actual wedge candidates, based on what you've built: (1) the effects-extraction/performance-audit craft, (2) the agent-memory/multi-agent tooling work, which is genuinely ahead of where most people are right now. Build the audience around demonstrating that specific depth — not around "look, I made another tracker app."

Mindset shift, concretely:

Stop spec'ing Ledger/Momentum like they're startups. Either timebox them hard as learning reps and ship fast/loose, or kill the 9-doc formalism on them — that ceremony is wasted on commodity apps.

Your actual differentiation is technical depth + taste (the extraction work, the agent tooling), not "I can also build a CRUD app." Redirect the spec-discipline and rigor you're putting into finance trackers toward documenting and publishing the extraction system and the agent architecture — those are things far fewer people can do and fewer still can explain.

"Game tester / UX auditor / product researcher" isn't a separate thing you need to go

find — it's the exact skill your EFFECTS_GLOSSARY/PLAYBOOK work already is. Name it, package it, show it publicly.

Do you need to figure this out now, or piece it together over time? Neither, cleanly. You don't need a five-year plan. But you do need to stop lying to yourself about what category your current projects are in — that's a now problem, because it's actively misdirecting your effort. The specific "important area" you end up known for can absolutely emerge over the next year of shipping. What can't wait is: stop over-investing rigor in disposable clone-projects, and start treating the craft-heavy work as the actual asset it is.



That's understandable. I encounter problems people face and build a walk around/fix. What I'm not doing is packaging those walk-arounds as products and shipping them instead. And publicly documenting my journey through building those workarounds.

I... don't know how to package something as a product. My repos are all public, partly because I point an agent to em, and because I want it to be visible. I guess if I package it as a product, no vibe coder is gonna look at the repo and say I want to clone this, they'll probably build from scratch and get a knock off instead. But again, when do I start posting about a build, while building and some goes to clone my repo and build it faster? Or when it's done? Or I when an mvp has been built (this is probably it). I asked what I should be focused on, so I need your help here cos my kinda not in the right place.

I haven't positioned myself mentally as someone who sells things to people, I think that's the biggest gap rn.

Side note(ice): I noticed on X, some apps/startups look to fix some of the issues indie hackers face, like marketing, code review, or something that they don't want to do themselves, and they make money off it cos most of the work is still in the hackers' hands and the app helps a lil bit for a subscription. In the end, the apps/startups get more money while the indie hackers' apps get less visitors/users because their apps don't solve much problems for people.

Show less

2:04 PM ↺ ✎ 📄 < 3/3 >

Diagnosed positioning gap and reframed selling as core opportunity

Let's take these one at a time, because a few of them are you avoiding the actual issue by asking a logistics question instead.

"When do I post — during, or at MVP?"

You already answered it yourself and then talked yourself out of it: "I guess ... they'll

probably build from scratch and get a knockoff instead." That's correct, and it means the clone-fear is mostly fake. Here's the actual mechanism: code is not the scarce thing anymore — you told me that yourself two messages ago, that's the whole premise of the Fireship/Levels argument. If someone can clone your repo, they could equally have vibecoded the same idea from a two-sentence prompt without ever seeing your code. Your repo being public changes essentially nothing about how fast a clone-happy competitor moves. What it does change is whether *you* get any credit, trust, or audience for having built it first and openly. So the answer is: post while building, from day one, not at MVP. Waiting for "done" just means you eat all the risk (nobody sees the work, nobody roots for it, nobody's there when it launches) for zero reduction in clone-risk. The "wait until it's polished" instinct is the trash idea here — kill it.

"I don't know how to package something as a product."

Concretely, packaging is the gap between "a workaround that works" and a thing a stranger can understand the value of in 10 seconds without reading code. It's four things, not more:

- A name and a one-sentence promise ("X for people who Y")

- A single clear entry point (a landing page, a README that reads like a pitch not a spec, a Loom demo — something a non-technical person could look at)

- A "before/after" — what pain existed, what it looks like solved. This is where you're actually strong, because you *find* real problems first (that's rarer than it sounds — most indie hackers invent problems).

- A way to say "here's how you get this" — even if it's just "DM me" or "here's the repo," there has to be an explicit offer, not just an artifact sitting there hoping to be discovered.

You already do 3. You're doing zero of 1, 2, 4. That's not a mystery skill — it's a habit you haven't built yet, and it's learnable in weeks, not years.

"I haven't positioned myself mentally as someone who sells."

This is the real gap, correctly identified, and it's bigger than it sounds. Notice the language you used throughout this whole conversation: "workaround," "fix," "build stuff and see results," "I like vibe coding." Every one of those is builder-brain, zero of them is offer-brain. You're describing a hobbyist's relationship to your own output — build it, admire it, move to the next problem. Selling isn't sleaze, it's just the discipline of finishing the loop: *this exists, here's who it's for, here's how you get it, please take it*. Right now you stop one step before that, every time, and call the unfinished loop "not knowing how to package." You know how — you're choosing not to close the loop because closing it means being seen asking for something, which feels different from being seen building something. That's the actual psychological thing to work on, not a tutorial you're missing.

Your side note is the sharpest thing you've said in this whole conversation — don't bury it.

You noticed something real: the meta-layer (tools that sell marketing/code-review/ops help to indie hackers) is capturing more value than the indie hackers' own products, because the meta-layer serves an exploding, permanent population (there will always be more new builders than there are winning products) while any single builder's app has to fight for a shrinking, cloneable niche. Look at what you've already built: a readme-generator skill, an agent-memory architecture, a multi-agent review pipeline, a design-extraction/effects system. That's exactly the meta-layer you just described — tools *for* builders, not another consumer app *competing with* builders. You're sitting on the more defensible side of the market and still spending your spec-discipline on a finance tracker. That's the clearest strategic misallocation in this whole conversation.

Where effort goes now, plainly ranked:

Start publicly documenting the design-extraction system and agent-tooling work *now*, mid-build, framed as "here's the problem, here's what I'm building to solve it" — not as a finished showcase.

Practice the close: every post ends with what it is, who it's for, and how to get it — even if "get it" is just "star the repo" today and "join the waitlist" in a month.

Treat Ledger/Momentum as reps, not businesses — timebox them, ship them loose, stop dressing them up with production-grade spec ceremony they don't need.

Sit with the discomfort of asking for something instead of just building — that's the actual practice, not a research task.

You don't need to have this fully figured out. You need to stop letting "I haven't figured it out yet" be the reason you never publish or never ask. Ship the next update this week with an actual "here's what this is for" line attached. That's the whole unlock.

